A ROS-Based Framework for Low-Cost Real-Time Haptic Feedback in Human-Robot Teaming

Martin Økter[1] and Filippo Sanfilippo[1]

[1]Department of Engineering, University of Agder, Norway, e-mails: martin.b.okter@uia.no, filippo.sanfilippo@uia.no

KEYWORDS

Human-Robot Teaming (HRT), Control, Haptics.

ABSTRACT

The evolution from human-robot interaction (HRI) to human-robot collaboration (HRC), and ultimately to human-robot teaming (HRT), represents a significant paradigm shift in how humans and robots work together. HRI initially framed robots as tools under human command, while HRC introduced greater autonomy in shared tasks. Today, HRT reimagines robots as equal team members, capable of adaptive communication and synchronized decision-making. A critical challenge in achieving effective HRT is developing haptic feedback systems that provide operators with real-time sensory information. This work presents a low-cost software-driven haptic feedback framework using ROS and micro-ROS to enhance real-time control in HRT environments. Unlike current costly, hardware-specific solutions, the proposed framework emphasizes accessibility and flexibility. System validation through precise force control tasks demonstrates the framework's potential to advance HRT through cost-effective, software-centric solutions.

INTRODUCTION

As industries embrace growing digitalisation, innovative methods for interacting with digital environments are increasingly vital. This progression mirrors the evolution from Human-Robot Interaction (HRI) to Human-Robot Collaboration (HRC), and ultimately to Human-Robot Teaming (HRT). Early HRI systems emphasised simple command-based interactions where robots acted as simple tools. The transition to HRC introduced collaborative tasks, with robots assisting humans through greater autonomy and adaptability. Today, HRT envisions robots as active team members, requiring advanced interfaces that enable intuitive, real-time interaction, and seamless integration into human workflows. Emerging technologies, such as mixed reality (MR) and haptics, are paving the way for more immersive and natural interactions, fostering deeper synergy between humans and digital systems.

For seamless integration into a digitally enhanced reality, the sense of touch plays a crucial role in human interaction with objects and robots, as it provides essential information about position and force (Fani et al., 2019). Unlike visual feedback, which is processed at a relatively lower temporal resolution, tactile feedback demands higher real-time responsiveness, optimally closer to 1 kHz (Sherman and Craig, 2018). Moreover, the extensive interaction area of tactile surfaces introduces complexities in delivering realistic and precise feedback, posing significant challenges for the development of effective haptic systems.

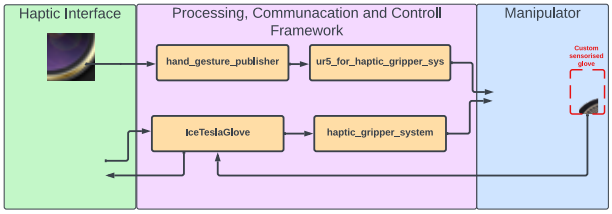

Figure 1: The proposed low-cost, software-driven haptic feedback framework for real-time HRT control.

Today, several haptic feedback devices exist, mostly developed as an extension used to interact with digital environments. Some commercial products for a robotic arm and a hand operating in a remote environment by an operator are available, but are usually a closed system from the manufacturer. One of the most famous examples and allegedly the first is the HaptX gloves controlled by Jeff Bezos at the re:MARS conference in 2019 (HaptX, 2019). Unfortunately, these products are very expensive and hardware-specific, and few have been used outside a research context. The specific used software framework are usually not open access, and only occasionally with an open application programming interface (API).

A working software framework that can dynamically be applied for multiple types of devices with small modifications could help further research of haptic control devices due to the low-effort implementation. By solving some major integration issues and researching compliant hardware components, a system can be set up and better hardware mechanics and control algorithms researched faster.

The contributions of this study are as follows:

- **Novel Framework**: introduces a software-driven haptic feedback framework for real-time control in HRT environments, asshown in Figure 1. The operator movement is collected through haptic sensory

Communications of the ECMS, Volume 39, Issue 1,
Proceedings, ©ECMS Marco Scarpa, Salvatore Cavalieri,
Salvatore Serrano, Fabrizio De Vita (Editors) 2025
ISBN: 978-3-937 436-86-9/978-3-937 436-85-2
(CD) ISSN 2522-2414

input, and then processed. The control values for the actuators are sent and executed. Environmental information are sent back for haptic feedback to the operator by the haptic wearable device.

- **Platform Integration**: ROS and micro-ROS to ensure accessibility, scalability, and modularity.
- **Sustainable solution**: overcomes the challenges of expensive, hardware-specific haptic solutions by emphasising cost-effectiveness and flexibility.

This work contributes to advancing HRT by fostering more intuitive, collaborative, and effective solutions. The source code for this project is available in four repositories: three for each core ROS node (*github.com/Microttus/qbshr-ctrl*, *github.com/Microttus/haptic_gripper_system*, *github.com/Microttus/ur5-for-haptic-gripper-sys* and one containing all edge/microcontroller code *github.com/Microttus/Ice-Gripper-Arduino-Code*).

RELATED RESEARCH WORK

Recent years, the advancement of virtual reality technology has provided new possibilities for an immersive experience with the help of wearable devices, such as the VR-Googles. This new technology has also made remote reality a possibility together with faster and more reliable internet communication. As Ayvasik et al. (Ayvaşık et al., 2023) show this technology can be used for successfully completion of tasks by control of a robotic device over substantial lengths. Here, a custom communication pipeline is created for control of a robot via a joystick and haptic input device. For a complete feel of immersiveness a natural way of interacting with the environment is necessary. In a recent paper Thakur et al. (Thakur et al., 2024) present a tetherless wearable haptic interface intended for remote robot control. With the use of a backpack with pneumatic control system and a artificial haptic muscle force feedback can be delivered to the operator. While the arm position is tracked with three IMU units placed on the operators arm. For increased sensitivity and feedback rending the quality of the force feedback using pneumatic actuation Yu et al. (Yu et al., 2022) developed a glove with self-sensing capability for force and vibration feedback.

Further necessary for a successful force feedback are the force feedback signal from the gripper side of the network. In his review of tactile sensing technologies for soft grippers, Qu et al. Qu et al. (2023) finds that even with the effort put into tactile sensing technologies for soft grippers, it still is at an infancy stage. A lot of the grippers on the market have no or little developed force feedback, which makes haptic remote control difficult.

Our research group previously introduced a flexible modelling and simulation architecture for haptic control of manipulators (Sanfilippo et al., 2013). Successively, an open-source library to couple a haptic device with a modelling and simulation environment was developed in (Sanfilippo et al., 2015). The presented middleware interface makes it possible to track the user's motion, detect collisions between the user-controlled probe and virtual objects, compute reaction forces and exert an intuitive force feedback on the user. A real-time one-to-one correspondence between reality and virtual reality can be transparently created. Recently, a review of the state-of-the-art of sensing and actuation technology for robotic grasping and haptic rendering was presented in (Moosavi et al., 2022). The potential of achieving multi-sensory learning through the integration of virtual and augmented reality (VR/AR) with haptic wearables in science, technology, engineering, and mathematics (STEM) education was investigated in (Sanfilippo et al., 2022).

To the best of our knowledge, there is limited research on developing a comprehensive, hardware low-cost framework for modular integration based on open-source and easy implementable software. A related effort was undertaken by Grabe et al. (Grabe et al., 2013), who developed a framework for haptic devices and unmanned aerial vehicle (UAV) control using ROS. However, their work relied on outdated frameworks, such as ROS version 1. Leveraging modern ROS and micro-ROS platforms, this research aims to establish a baseline for further studies on haptic feedback control structures. Additionally, it provides initial testing to evaluate latency and performance, contributing insights for future advancements.

HARDWARE DESIGN

The system requirements prioritize modularity and research functionalities. The system components must operate independently while allowing for expansion. As a research tool for studying object handling forces and movements, the framework must support data collection. The key requirements are as follows: a lightweight structure ensuring portability; accurate force and position measurement; precise force feedback for intuitive interaction; a safe workspace protecting the operator; modular design for flexible implementation; comprehensive data collection capabilities.

Hardware system overview

While building a test platform, the goal is to demonstrate the system's potential without unnecessary complexity. Remote gripping of objects with varying stiffness requires precise force control to ensure firm grip without damage. The setup employs a qbrobotic SoftHand Research (qbrobotic, 2024) and a Universal Robot 5e (UR5e) (Robots, 2024). For operator control, a custom wearable glove is developed to enable finger tracking and force feedback delivery.

As a modular design is set as one of the requirements, the framework connecting the hardware interfaces is built using several nodes and individual components. Referring to Figure 1, the hand position is collected using a web camera, and an ROS2 node that analyses the picture. A position is measured and posted on a topic for the robot arm node to read. The new position is processed by the robot arm node and sent to the robot for actuation. As the UR5e robot arm is included as a library, minimal effort

would be required to replace the parts relating to machine-specific code.

Furthermore, the "IceTeslaGlove" node is used to collect data from both the force sensor pads attached to the robotic arm and the position of the operator's hand. These data are processed in such a way that appropriate feedback can be provided to the operator's glove. The glove is developed as a low-cost device, and has servos attached to a 3d-printed ergonomic plate strapped to the palm of the operator and connected to the fingertips. Hence, adaptive force feedback recorded by the force pads can be processed and applied to the operator's hand, see Figure 2. In addition, the current grip state of the operator is processed and sent to the "happtic_gripper_system" node. This node has the robotic hand implementation and is responsible for setting up the communication to the hand using RS485. This is done using the custom ROS2 package, which implements the *qbrobotic SoftHand* API.

The system operates within the ROS2 framework, enabling the execution of nodes on any computer within the same network. However, challenges arose with the Fast-DDS XRCE protocol used for communication between ROS2 and micro-ROS. Specifically, issues such as topics becoming inaccessible on other computers after microcontroller reconnections were encountered. Additionally, the laptop designated for camera tracking experienced difficulties running the system effectively. To address these issues, the camera tracking and UR5 client are executed on one computer, while the micro-ROS agent, *IceTeslGlove*, and *haptic_gripper_system* are deployed on an older computer. This configuration provides a stable environment for running the micro-ROS agent reliably.

Due to the decision to use micro-ROS for the microcontroller to integrate into the ROS2 system, the choice of microcontrollers is limited. From the GitHub documentation of the mico_ros_arduino package, a selection of mostly high performing microcontrollers is listed. Among them, the ESP32 Dev Module is listed as officially supported.

Maintaining the human-in-the-loop (HITL) necessitates a high update frequency to ensure seamless interaction. Research indicates that an update rate of at least 1 kHz is essential to prevent the system from feeling "sluggish" or exhibiting delayed responses. Since the goal of this work is to provide the operator with a tactile experience that closely resembles directly handling the object, smooth and responsive operation of the hardware is crucial. Consequently, the choice of microcontroller is heavily influenced by its core frequency to meet these performance demands.

Figure 2: The sensorised glove and feedback device.

micro-ROS nodes are able to communicate with an ROS2 environment via Wi-Fi or Ethernet. Since wearable devices should be minimally invasive for the user's workspace, a wireless connection is preferred. The downside is the reliance on continuously working and low-latency Wi-Fi, introducing a single point of failure to the system. Wi-Fi add-on boards are available but would require more cables and result in poorer mounting. An integrated Wi-Fi connection is therefore strongly preferred.

Due to the requirements for this project, especially the network, support, cost and versatility, the TinyPICO (Rozenblum, 2025) is selected as the main microcontroller platform for implementation.

Robotic Arm

The selection of the robotic arm is guided by specific criteria, including compatibility with the *qbrobotic SoftHand*, lifting capacity, and operational ease. The *qbrobotic SoftHand* research model is supplied with tools and adapters designed for integration with a selection of robotic arms, such as *Universal Robots*. With the *SoftHand* weighing approximately one kilogram and the intended application involving tasks such as lifting groceries, a robotic arm with a lifting capacity of at least two kilograms is deemed necessary. Several robotic arm options were systematically evaluated based on their technical specifications, availability, and ease of integration with the system.

Universal Robots has both an official ROS2 support library, and a 3rd party version. The Real-Time Data Exchange (RTDE) library is developed by the University of Southern Denmark (of Souther Denmark, 2024), and provides features such as pre-made inverse kinematics.. It is built as a C++ library API and can easily be integrated. It handles all communication with a specified UR robot connected to the same local network. Considering the ease of use and capability to lift up to 5 kilograms, an UR5 arm is chosen as the platform for the system.

SOFTWARE DESIGN AND INTEGRATION

Referring to Figure 3, the software is organised into two logical layers: the low-level logical layer, and the high-level logical layer. The low-level part of the system contains several nodes which are deployed on physical hardware, such as microcontrollers and actuators. This part of the code is developed using Arduino (Arduino, 2025) and the Arduino language as a tool, and is mainly used for collecting sensor data and deploying actuator commands. For communication, two main protocols are adopted. For internal communication between microcontrollers, the Inter-integrated circuit (I2C) protocol is used. This protocol is supported by most hardware and has high flexibility and bandwidth. For communication between edge nodes in the low-level layer and into the main core system, micro-ROS is adopted. micro-ROS is a framework which, through a

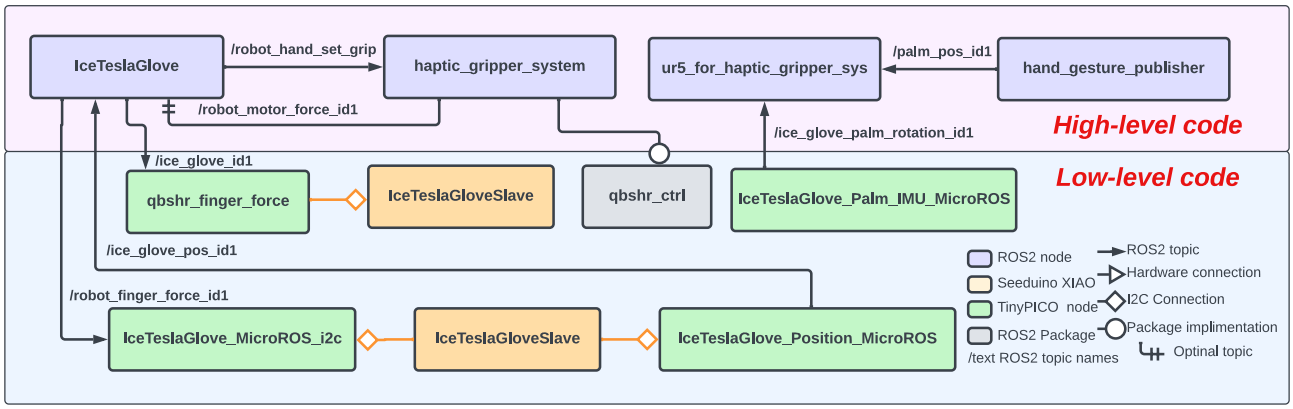

Figure 3: An overview of the setup of the system with the high-level layer, based on ROS2 nodes, and the low-level layer.

custom pipeline, enables Data Distribution Service (DDS) communication for connection to an ROS2 system.

The high-level layer is the main computational layer of the system. The core is built for an ROS2 framework, enabling DDS communication between the dedicated computational nodes and also by taking advantage of micro-ROS out to the edge low-level nodes. The ROS2 framework supports a wide range of programming languages, which is beneficial for a system that should be able to handle multiple types of tasks. For instance, Python makes object detection integration easy by the use of ready-made libraries, while C++ has superior speed and reliability for real-time computation.

By organising the proposed framework into two layers, a more robust and versatile system is created. Edge nodes have a single-task nature and hence the loop time can be kept short, maintaining real-time properties. Heavy calculation and control are centralised in the core, which can be run on a more powerful central hub. The nature of the system also allows for easy addition of extra edge nodes for increased capabilities.

In Figure 3, the purple boxes are ROS2 nodes running on the control computer, the green boxes are micro-ROS nodes running on TinyPICO chips, and the orange ones are a program running on a Seeduino XIAO accessed with an I2C connection. The orange boxes represent the implemented packages. The normal black arrows are ROS2 topics, the big arrows are hardware connections (such as USB), the orange square arrows I2C, and round package implementation. The double-lined arrows are optional information.

Low-level Logical Layer

The low-level layer of the system consists of a collection of individual edge nodes for the collection of data and actuator control. Due to the nature of running on microcontrollers with limited processing power, minimising the complexity of single node tasks is important to retain real-time properties. As micro-ROS is used for communication and system framework, templates for building micro-ROS nodes on microcontrollers based on the Arduino framework were used. The micro-ROS project provides the codelines needed to set up the connection to the agent on the host computer, as well as to port the information needed about the topics and nodes. This makes the three separate hardware codes quite similar in terms of setup. Most of the code is directly tied to micro-ROS and only the names of nodes and topics are changed as well as minor changes to ports and data conversion.

Working with micro-ROS implementing new message types is especially challenging, hence a standard available message is chosen. In particular, the Twist message in the geometry_msgs package is selected, as this message has 6 data fields which are directly addressed by name. This is important, as the micro-ROS based Arduino code seems to not handle dynamic message-types very well, possibly leading to core panic.

Given that human hands have five fingers, a message type with five fields is well-suited to the current state of the hardware. The standard twist message, which offers six data fields, can accommodate this design, with the sixth field reserved for potential future integration of palm force feedback. For further optimisation, a custom message type specifically tailored to this application could be developed, enabling enhanced alignment with the system's unique requirements.

Listing 1: micro-ROS Wi-Fi setup code

```
void setup() {
// Connect to Wifi
  set_micro_ROS_wifi_transports("
      SSID_name", "SSID_pwd", "IP_addr
      ", 8888);
```

For the main section of the micro-ros setup code, the following procedure was used. First, as shown in the code section above, the Wi-Fi connection is configured for the preferred Wi-Fi and the IP address of the host computer. Additionally, a port which is used for the connection is provided, as standard port 8888 is used. An allocator is created and used to establish a node, which is then connected to a subscription. For the subscription, a message type is provided. Finally, an executor is created to ensure the callback function is called whenever updates on the

topic are available. If a publisher is used, a timer is created and the loop function is employed as the function to repeat at the requested frequency.

Finger Position Edge Node:

A dedicated node is implemented to track the bending states of the operator's fingers. A microcontroller, connected to positional sensors mounted on the fingers, is programmed to read/transmit data upon request via the I2C protocol. This data is then processed by another microcontroller running a lightweight micro-ROS publisher. The publisher periodically retrieves the sensor data and publishes it on a topic at regular intervals, ensuring consistent communication and integration within the system. During testing, it was found that the initialisation of the Wi-Fi module on the TinyPICO would deactivate the analogue-to-digital-converter (ADC), making readings of the load cells impossible. For further development, a replacement chip to the pico should be investigated. As a workaround, the introduction of a dedicated node reading the data and delivering on request, as shown in Figure 3 as "IceTeslaGloveSlave", was introduced.

IceTeslaGlove Edge Node:

The IceTesalaGlove node manages the control of the servo motors connected to the palm plate and the operator's fingertips through mechanical links. This edge node is deployed on a microcontroller embedded in the operator's glove and operates using micro-ROS control code, with two versions available. The simplest version primarily retrieves servo position data from an ROS2 topic and applies these positions to the servos. This minimal implementation consists of a basic micro-ROS subscriber that reads a Twist message and directly adjusts the servo positions based on the received values.

Fingertip Force Edge Node:

As the robotic hand chosen for the setup does not provide single finger sensor feedback natively, a sensorised glove is constructed. By adding force sensors for each fingertip connected to a microcontroller, force information is provided for the system. The layout of this node closely imitates the one used for measuring the position of the operator's fingers. A dual setup with one TinyPICO connected to a Seeeduino XIAO is used, setting the XIAO up as a slave providing analogue readings on demand.

The setup for this code is intended to provide a versatile platform for gathering multiple different types of signals and publishing to an ROS2 topic. This way, the setup and code could easily be set in a place where the sensor data is wanted, connected, and used. Due to the ADC issue, the dual setup is needed and complicates the process.

High-level Logical Layer

The core part of the system is supposed to handle main computational tasks in the system and collect data from sensor nodes as well as provide control signals for actuator nodes. For the current system, the high-level layer consists of four different nodes which is launched separately in an ROS2 framework providing the communication layer. The architecture is meant to be set up with one node handling incoming signals, a transfer topic, and a node converting them into control signals for the device. For the control of the robotic arm, communication is quite consistent. A node is used to process the camera data and post the position of the palm on a topic. Another node reads the position and controls the robotic arm to the position of the tool according to the request.

This layout with one node handling input and another output makes it easy to change out hardware, as only the corresponding node needs to be updated or replaced. The "middle" topic still represents the same information. For example, it could be possible to change the robotic arm to one of a different brand. In such a case, the control library only needs to be altered. Or if no physical implementation is used, only a simulation in, for instance, the Gazebo Simulator (Robotics, 2025), a new output node able to convert the position topic to gazebo position is needed.

The node handling the control of the developed haptic glove, called *IceTeslaGlove*, has several connections as it requires information for various parts of the system for control. Due to the connection of the hardware, especially the robotic hand, both receiving and sending communication to the same device is handled by a single node. The nodes are more connected to the actual hardware, as one node both controls and measures the glove aspect, while another controls and monitors the robotic hand.

qbrobotics SoftHand research control package:

This package called *qbshr_ctrl* is developed such that the qbroboitic Soft Hand Research library could be implemented into an ROS2 environment. The manufacturer provides an API which enables full control of the robotic hand at a raw code level. From this API, two classes are used for instantiating an object, *qbSoftHandControl* and *qbSoftHandHandler*. The handler is responsible for detecting available hands connected to the computer and setting up a communication line to each of them under a specific flag. The flags can then be handed to the control section for a specific command.

This package is developed with making integration into future projects as easy as possible. Hence, it is compiled as a library and implemented as an ROS2 package. This package can be added to the CMake file of the ROS2 package where use is intended. Another approach would be a more direct importation of the API in the package, but this would make changes of hardware much more difficult, and as a result, lower the adaptability of the code.

Haptic Gripper Control Package:

The *haptic_gripper_system* implements the created *qbshr_ctrl* package and sets up communication with the created ROS2 environment. The main aspect related to the haptic glove, called IceTeslaGlove, is the inspiration of the tesla glove. The information is read from the topic

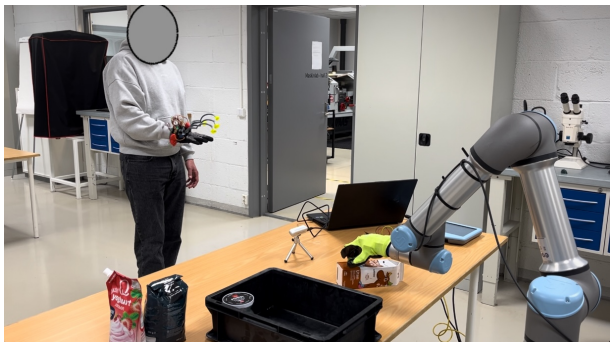

Figure 4: The operator on the left wearing the haptic glove and the robotic arm with the robotic hand on the right.

on which the position of the finger is posted, in addition to the force. If the sensorised glove is used, the topic published from this microcontroller is used; if not, the one approximated by the robotic hand node could be used.

The package allows for a range of different approaches for the calculation of the feedback system, enabling the user to adjust the node according to the limitations of the hardware. For instance, if the internal I2C loop is not activated for the low-level setup, the output position is fully calculated within the range of the full finger work area. If the I2C feedforward loop is used, only a correctional value is calculated and provided to the feedback device. Also, the range controlling the amount of feedback can be adjusted in the code. Although feedback is calculated, this value can be ignored by the feedback device if the total value is outside the work area.

For the robotic hand part, the amount of grip is calculated on the basis of a normalisation of the read finger position topic, where an average over the finger is used. Since finger positions cannot be controlled individually, this is a compromise to ensure that the robotic hand grips the object even though one finger is not able to move due to object contact.

Robotic Arm Control Package:

The robotic arm control related to the project is connected and controlled to this package which is instantiated by the system as *ur5_for_haptic_gripper_sys*. As the Universal Robot 5e was chosen for the project, the RTDE library was leveraged for control. As the RTDE already provides robust control, the integration of the controller could be made quite simple. A single node was set up to read the position from the topic and restructure it into the correct position structure, and further sent to the robotic arm with a *moveJ_IK* command. This tells the library to use its built-in inverse kinematics for calculating joint movement based on the Cartesian goal coordinate input.

EXPERIMENTS

To evaluate the complete system, an experimental setup is designed comprising a Universal Robot 5e robotic arm, an Intel RealSense D435i webcam for hand tracking, and a qb Robotics Soft Hand Research model. The experimental setup is illustrated in Figure 4. The robotic hand is mounted onto the robotic arm, which is positioned behind a table. To ensure safety, the robotic arm's movement is restricted to the xz plane, minimising the risk of collisions with itself, unintended objects, or observers. The webcam is placed at the edge of the table, oriented towards the operator's workspace and away from the robot. The operator is equipped with the developed haptic glove device and positioned in front of the camera. Following system initialisation, a calibration process is conducted to adjust for operator moveability and ensure optimal performance.

To emulate the variable stiffness control naturally used by humans, a method for controlling a gradual change in position to apply force to an object is required. While humans interact with objects in this manner instinctively, a controller capable of replicating this behaviour for a robotic hand is necessary. To this end, an impedance controller is employed similar to the implementation by Sanfilippo et al. (Sanfilippo et al., 2024) in their open-source elastic actuator. The impedance controller relates the force and position of a system by modelling it as a spring-damper system, thereby providing the desired system properties. An initial tuning of the system was conducted with basic values for testing.

To validate the impedance control approach, a recording of the reported force and the calculated position offset feedback is made. This test relies on the full input/output line from the robotic hand to the operator glove actuation to function. In Figure 5, weights of 15 gram are successively added to one of the resistive force sensors, and then removed one by one. The weight is recorded as a force by the edge node running on an XIAO controller and published to the force topic by the *qbshr_finger_force*. The published message is picked up by the control ROS node IceTeslaGlove for processing. The recorded force is calculated to a position by an impedance tuned controller, before being published to the finger control topic. Another edge node retrieves the published message by the use of micro-ROS and send the analogue signals to the final microcontroller for servo actuation. Referring to Figure 5, the blue line is the normalised force reported by the sensors, and the orange line is the calculated feedback signal sent to the glove. As it can be observed, the blue line steps up in some sections, and also the non-linearity of the system can clearly be observed. For this experiment, an impedance using a spring value of 30 N/m and zero for mass and damper is used. The system is responsive and follows the input signal. As this process shows, the values are collected at one end of the program, published by micro-ROS, processed by ROS, then sent to a micro-ROS node for execution. The total delay time is shown as the divergence between the lines, and was estimated with cross-correlation to be approximately 107 ms.

Figure 6 illustrates the system setup during the interaction with an object. The orange line, representing the feedback signal, closely follows the force input signal, albeit with a slight observable delay. The filtering effect

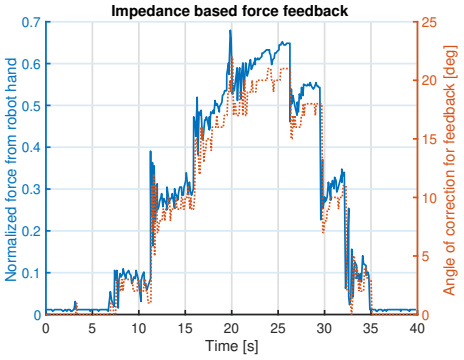

Figure 5: Test of impedance response adding weights.

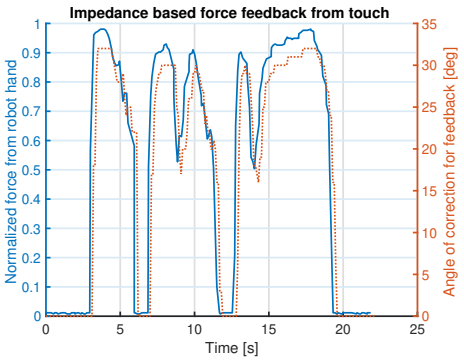

Figure 6: Test of impedance response touching an object.

of the current setup is evident, as the orange line demonstrates a smooth response without minor oscillations.

The feedback force delivered to the operator, represented as the control signal angle, demonstrates a strong response and correlation with the sensed force. For both this test and subsequent operator trials, a basic tuning approach was employed, resulting in a fast system with minimal damping of inertia. As illustrated in Figure 6, the current tuning provides realistic feedback for the system, especially given the limited range of the force sensors and the significant force required to handle system components. However, as shown in Figure 5, oscillations are observed when the force sensor is gradually loaded. These oscillations could be further mitigated by introducing additional damping to the system. Considering that human fingers typically exhibit rapid movement with low inertia relative to the available power, prioritising an inertia constant is deemed unnecessary.

CONCLUSION

This work focused on developing a low-cost software-driven haptic feedback framework to enhance real-time control in human-robot teaming (HRT) environments. The proposed system integrates Robot Operating System (ROS) 2 and micro-ROS platforms, taking advantage of their modularity and flexibility to enable effective HRT. The system was designed to provide human operators with real-time sensory information about force and position, enhancing intuitive teamwork. Hand detection and robotic arm control were implemented purely as ROS2 applications, while sensor data acquisition and actuation were controlled by microcontrollers running micro-ROS. Although micro-ROS initially did not meet the required frequency and reliability, with some adjustments and modifications, it provided a functional and scalable framework, demonstrating the potential for cost-effective, real-time haptic feedback in HRT environments. The framework was validated through tasks that require precise force control, such as manipulating objects with varying material properties, demonstrating its potential to improve haptic control mechanisms and advance HRT.

Future work could look at how to replace micro-ROS with a system with the same capabilities, such as wireless connection and ROS compatibility, but with added responsiveness and reliability. Also, standardising the message formats and possibly making custom more suitable messages for the pipelines could help make the framework even more flexible and dynamic. Furthermore, other different types of controllers such as reinforcement learning could be explored.

ACKNOWLEDGEMENT

This research is supported by the Artificial Intelligence, Biomechatronics, and Collaborative Robotics research group at the Top Research Center Mechatronics (TRCM), University of Agder (UiA), Norway.

REFERENCES

Arduino. Arduino, 2025. URL https://www.arduino.cc/.

Serkut Ayvaşık, Edwin Babaians, Arled Papa, Yash Deshpande, Alba Jano, Wolfgang Kellerer, and Eckehard Steinbach. Remote robot control with haptic feedback over the munich 5g research hub testbed. In *Proc. of the 24th IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, pages 349–351, 2023.

Simone Fani, Katia Di Blasio, Matteo Bianchi, Manuel Giuseppe Catalano, Giorgio Grioli, and Antonio Bicchi. Relaying the high-frequency contents of tactile feedback to robotic prosthesis users: Design, filtering, implementation, and validation. *IEEE Robotics and Automation Letters*, 4(2):926–933, 2019.

Volker Grabe, Martin Riedel, Heinrich H. Bülthoff, Paolo Robuffo Giordano, and Antonio Franchi. The telekyb framework for a modular and extendible ros-based quadrotor control, 2013. URL http://coppeliarobotics.com.

HaptX. Jeff bezos operates a tactile telerobot, 2019. URL https://www.youtube.com/watch?v=uwYtwQtoOh0&ab_channel=HaptX.

Syed Kumayl Raza Moosavi, Muhammad Hamza Zafar, and Filippo Sanfilippo. A review of the state-of-the-art of sensing and actuation technology for robotic grasping and haptic rendering. In *Proc. of the 5th IEEE International Conference on Information and Computer Technologies (ICICT), New York City (virtual), United States*, pages 182–190, 2022.

University of Souther Denmark. Universal robots rtde c++ interface, 2024. URL https://sdurobotics.gitlab.io/ur_rtde/.

qbrobotic. qb softhand research, 2024. URL https://qbrobotics.com/product/qb-softhand-research/.

Juntian Qu, Baijin Mao, Zhenkun Li, Yining Xu, Kunyu Zhou, Xiangyu Cao, Qigao Fan, Minyi Xu, Bin Liang, Houde Liu, et al. Recent progress in advanced tactile sensing technologies for soft grippers. *Advanced Functional Materials*, 33(41):2306249, 2023.

Open Robotics. Simulate before you build, 2025. URL https://gazebosim.org.

Universial Robots. Ur5e, 2024. URL https://www.universal-robots.com/products/ur5e/.

Seon Rozenblum. Tinypico, 2025. URL https://www.tinypico.com/.

Filippo Sanfilippo, Hans Petter Hildre, Vilmar Æsøy, Houxiang Zhang, and Eilif Pedersen. Flexible modeling and simulation architecture for haptic control of maritime cranes and robotic arm. In *Proc. of the 27th European Conference on Modelling and Simulation (ECMS), Aalesund, Norway*, pages 235–242, 2013.

Filippo Sanfilippo, Paul B.T. Weustink, and Kristin Ytterstad Pettersen. A coupling library for the force dimension haptic devices and the 20-sim modelling and simulation environment. In *Proc. of the 41st IEEE Annual Conference of the IEEE Industrial Electronics Society (IECON), Yokohama, Japan*, pages 000168–000173, 2015.

Filippo Sanfilippo, Tomas Blazauskas, Gionata Salvietti, Isabel Ramos, Silviu Vert, Jaziar Radianti, Tim A Majchrzak, and Daniel Oliveira. A perspective review on integrating vr/ar with haptics into stem education for multi-sensory learning. *Robotics*, 11 (2):41, 2022.

Filippo Sanfilippo, Martin Økter, Jørgen Dale, Hua Minh Tuan, Muhammad Hamza Zafar, and Morten Ottestad. Open-source design of low-cost sensorised elastic actuators for collaborative prosthetics and orthotics. *HardwareX*, 19:e00564, 2024.

William R. Sherman and Alan B. Craig. *Understanding virtual reality: Interface, application, and design*. Morgan Kaufmann, 2018.

Shilpa Thakur, Nathalia Diaz Armas, Joseph Adegite, Ritwik Pandey, Joey Mead, Pratap M Rao, and Cagdas D Onal. A tetherless soft robotic wearable haptic human machine interface for robot teleoperation. In *Proc. of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12226–12233, 2024.

Meng Yu, Xiang Cheng, Shigang Peng, Yingze Cao, Yamei Lu, Bingyang Li, Xiangchao Feng, Yan Zhang, Haoyu Wang, Zhiwei Jiao, et al. A self-sensing soft pneumatic actuator with closed-loop control for haptic feedback wearable devices. *Materials & Design*, 223:111149, 2022.

AUTHOR BIOGRAPHIES

MARTIN ØKTER received his master's in Mechatronics from the University of Agder in 2024 and is currently a Doctoral Research Fellow at UiA. He has made several contributions to the courses taught in the Mechatronics programme, in addition to the development of laboratory equipment for teaching haptic control theory for graduate and postgraduate students at the Haptic Summer School (EMERALD). His primary research interests include applied control theory for collaborative robots (cobots), adaptive RL robotic control, and haptic feedback. He is also a member of the Norwegian Society of Mechatronics (NSOM). Furthermore, one of his publications received the Best Paper Award.

FILIPPO SANFILIPPO holds a PhD in Engineering Cybernetics from the Norwegian University of Science and Technology (NTNU), Norway. His research focuses on Human-Robot Teaming (HRT). He is a Professor at the Faculty of Engineering and Science, University of Agder (UiA), Norway, and the Head of the Artificial Intelligence, Biomechatronics, and Collaborative Robotics research group. He is an IEEE Senior Member, former Chair of the IEEE Norway Section, and currently Chair of the IEEE Robotics and Automation, Control Systems, and Intelligent Transportation Systems Joint Chapter. He also leads the Norway Section Life Members Affinity Group and serves on several IEEE Region 8 committees, including Chapter Coordination, Conference Coordination Committee, Public Visibility, Awards & Recognitions, and Professional & Educational Activities. He has published numerous technical papers in leading journals and conferences and actively serves as a reviewer for international publications.